

Phase 2 — Explained

Agent core: account pool & isolated browser sessions

27 July 2026 · Phase 2 of 11 · Status: **ALL CHECKS PASSED LIVE**

Contents

1. What Phase 2 was for
2. Every file created or edited
3. What was tested live, and the results
4. The bug found and fixed
5. Key concepts
6. Decisions made
7. What you should do right now
8. What's needed before Phase 3

1. What Phase 2 was for

The build plan's goal for this phase: *"the agent can open isolated browser sessions per account, each on its own proxy."* This is a foundation phase, not a features phase — it builds the safety mechanism every later phase depends on. Get this wrong, and Phase 3 (discovery) or Phase 7 (sending) could leak one account's session or cookies into another's, which is exactly what gets real LinkedIn/Instagram accounts flagged or banned.

Four things had to be true by the end: an account pool manager that knows which of the 3 accounts should run and when; a session mechanism that opens a genuinely isolated browser context per account; a health check that detects trouble with a specific type and reason rather than a generic failure; and a scheduler that ties it together, respecting each account's own run time and never auto-redistributing a paused account's capacity.

All four are true and proven with real, live tests — not simulated. Every claim in this document was actually run against your Supabase project and a real Chromium browser, with the evidence shown in section 3.

2. Every file created or edited

`agent/db/repositories.py` EDITED

Added one function, `has_run_today(account_id, day_start_iso)` — checks whether an account already has a `runs` row starting today, so the scheduler never dispatches the same account twice in one day. Extends the Phase 1 data-access layer rather than creating a new one, keeping the "one file touches the database" rule intact.

`agent/core/account_pool.py` NEW

Role: decides which accounts should run right now.

Logic: an account is "due" when it is `active` (never `paused` — a paused account never un-pauses itself; that is Hussein's call, core rule R9), its configured `run_time` has already passed in today's local time, and it has not already produced a run today. A `force=True` override skips the time and once-per-day checks entirely — this is what manual testing uses, so a real run can be triggered any time of day without waiting for an account's actual scheduled hour.

`agent/core/session.py` NEW

Role: the actual isolation mechanism — the most safety-critical file in this phase.

Architecture: one shared Chromium *process* is launched for the whole run (cheap), and each account gets its own Playwright *context* — an isolated profile with its own cookies, storage, and cache, comparable to a separate incognito window that never shares anything with another. `SessionManager` owns the shared browser and hands out these per-account contexts.

Two extra things built beyond the bare minimum:

- **Per-account proxy binding** — `build_proxy_config()` reads an account's `proxy_host` / `proxy_port` / `proxy_username` / `proxy_password` columns and returns a Playwright proxy dict, or `None` if none is set (the case for all 3 accounts today — proxies are a Phase 10 concern).
- **Persistent per-account storage** — each account's cookies/login state are saved to `agent/browser_profiles/<account_id>.json` after every session and reloaded at the start of the next one. This matters for real safety: logging in from scratch every single day is itself a signal platforms watch for; staying logged in between runs looks like normal human behaviour. That folder was already gitignored back in Phase 0, which turned out to anticipate exactly this correctly.

agent/core/health.py NEW

Role: decides whether a session hit trouble, and records exactly what, never a generic failure.

Two detection layers:

- `check_navigation()` — did the page even load? Catches a missing response (DNS/timeout/connection failure), HTTP 403 (blocked), HTTP 429 (rate limited), or any other 4xx/5xx. Fully testable today against neutral pages, and was tested (section 3).
- `check_page_content()` — scans the loaded page's text for known challenge/CAPTCHA phrasing (`CHALLENGE_PHRASES` : captcha, login_challenge, checkpoint, rate_limited, each with real platform wording). This does not fire on the neutral test pages used in this phase, by design — it exists now, fully built, so Phase 3 can call it unchanged against real LinkedIn/Instagram pages without rebuilding this logic later.

`record_warning()` pauses the account and logs the type + reason via the `set_account_warning` function already built in Phase 1. It deliberately never touches `redistribute_flag` — core rule R9 says redistribution is a manual decision, so this module only ever reports a problem, never acts on it.

agent/scheduler.py NEW

Role: ties the previous four pieces together into one runnable cycle.

`run_cycle(target_url, force)` — for each due account: `start_run()` (opens a `runs` row) → open an isolated session → navigate to the target → run both health checks → close the session (saving its storage) → either `finish_run(status="completed")` or `record_warning()` + `finish_run(status="error")`.

`build_daily_schedule(accounts)` — wires one APScheduler cron trigger per account at its own `run_time`, in the project's configured timezone. This is real, callable code, but is not exercised during local development, since nothing here runs a permanent 24/7 process yet — that begins in Phase 10.

Running the file directly (`python -m agent.scheduler`) triggers one manual cycle immediately against a neutral test page — this is what "Hussein can trigger a run" means for this phase.

3. What was tested live, and the results

Test 1 — session isolation (the core safety property of this phase)

Opened 3 real, simultaneous browser contexts, one per seeded account, each navigating to `example.com` and setting a cookie carrying its own account id. Then read cookies back from every context to check for cross-contamination.

```
testing isolation across 3 accounts

opened isolated context for Account 1 (2fe03e3f...)
opened isolated context for Account 2 (2ad062fb...)
opened isolated context for Account 3 (948dc7e2...)

checking each context only sees its OWN cookie:
PASS  Account 1: sees cookie value(s) ['2fe03e3f-f5c1-48fe-93ad-1981f2f5e995']
PASS  Account 2: sees cookie value(s) ['2ad062fb-f24d-4857-a6af-649d29f070e8']
PASS  Account 3: sees cookie value(s) ['948dc7e2-a2d2-40c3-aecc-853a680f1c76']

ALL ISOLATED - no cross-account leakage
```

Also confirmed: each account's persistent storage file was written correctly (one JSON file per account id in `agent/browser_profiles/`) and every one is gitignored — checked with `git check-ignore` before anything was left on disk longer than the test.

Test 2 — the success path, against real Supabase

`python -m agent.scheduler` (forced, ignoring the real schedule):

```
Manual test cycle -- 2026-07-27 01:43:31
Target: https://example.com

Account 1: OK
Account 2: OK
Account 3: OK
```

Confirmed directly against the live `runs` table afterward — real rows, with a real start time and, after the fix in section 4, a real finish time:

```
948dc7e2 | completed | started: 2026-07-26T22:43:37... | finished: 2026-07-26T22:43:38...
2ad062fb | completed | started: 2026-07-26T22:43:35... | finished: 2026-07-26T22:43:37...
2fe03e3f | completed | started: 2026-07-26T22:43:34... | finished: 2026-07-26T22:43:36...
```

Test 3 — the failure path (a deliberately broken target)

Ran the same cycle against a domain that does not exist, to prove `check_navigation()` genuinely catches a real failure rather than only a hypothetical one:

```
Account 1: WARNING (navigation_failed)
  reason: Page.goto: net::ERR_NAME_NOT_RESOLVED at https://this-domain-does-not-exist...

Account 2: WARNING (navigation_failed)
  reason: Page.goto: net::ERR_NAME_NOT_RESOLVED at https://this-domain-does-not-exist...

Account 3: WARNING (navigation_failed)
  reason: Page.goto: net::ERR_NAME_NOT_RESOLVED at https://this-domain-does-not-exist...
```

Then confirmed against the live `accounts` table that all 3 were correctly paused with a specific type and reason:

```
Account 1 | status: paused | warning_type: navigation_failed | reason: Page.goto:
net::ERR_NAME_NOT_RESOLVED...
Account 2 | status: paused | warning_type: navigation_failed | reason: Page.goto:
net::ERR_NAME_NOT_RESOLVED...
Account 3 | status: paused | warning_type: navigation_failed | reason: Page.goto:
net::ERR_NAME_NOT_RESOLVED...
```

And that `redistribute_flag` stayed `False` on every account — proving the agent never acts on a warning beyond pausing and logging it:

```
Account 1 | redistribute_flag: False
Account 2 | redistribute_flag: False
Account 3 | redistribute_flag: False
```

Cleanup: since this was a deliberate test rather than a real problem, all 3 accounts were reset to `active` with warnings cleared, the 6 test `runs` rows were deleted, and the local browser profile files were removed — your real database and local machine were left exactly as they should be for Phase 3 to begin cleanly.

4. The bug found and fixed

Symptom: after the first successful test run, every `runs` row showed `status: completed` but `finished_at: None`.

Cause: `repositories.finish_run()` only writes the `finished_at` column if a `finished_at_iso` argument is explicitly passed in — it has no automatic "now" default the way `started_at` does in the schema. `scheduler.py`'s `run_cycle()` was calling it without that argument, so the column was silently never set.

Why this mattered: the entire purpose of the `runs` table is finish-time tracking — it feeds the dashboard's Run Status panel and the exact duration quoted in the daily WhatsApp summary (both future phases). A silent `None` here would have surfaced much later as a confusing missing-data bug in Phase 9, far from its actual cause.

Fix: `run_cycle()` now computes `dt.datetime.now(dt.timezone.utc).isoformat()` once per account and passes it explicitly to both the success and failure branches of `finish_run()`. Verified by re-running the cycle and reading the real rows back — see the corrected output in section 3, Test 2.

5. Key concepts

Browser process vs. browser context

A Playwright *browser* is one running Chromium process. A *context* is an isolated profile inside that process — its own cookies, storage, cache, and (as used here) its own proxy. Launching 3 separate browser processes for 3 accounts would work too, but costs far more memory and startup time for no extra isolation benefit — contexts already give complete separation. This is the standard pattern for exactly this kind of multi-account automation.

Why persistent storage state matters for account safety

Without saving a context's `storage_state`, every run would start from a completely blank browser — no cookies, no login. That means re-authenticating from scratch every single day, which is itself a pattern real platforms watch for. Saving and reloading each account's storage file means, once real logins happen in a later phase, an account looks like it stayed logged in between sessions — the same as a real person leaving their laptop open.

Why the account pool never un-pauses an account

This is core rule R9, and it is enforced structurally, not just by convention: `get_due_accounts()` filters to `status == 'active'` only, and nothing anywhere in this phase's code ever writes `status` back to `'active'` or touches `redistribute_flag`. The only way a paused account starts running again is a human changing its status from the dashboard (Phase 9) — the code has no path to do it itself.

6. Decisions made

Decision	Reasoning
Tested against neutral pages (<code>example.com</code>), not real LinkedIn/Instagram	Hussein's explicit choice — proves the mechanism with zero risk to real accounts. Real platform interaction begins in Phase 3, once discovery logic exists to justify it.
Built <code>check_page_content()</code> 's real challenge phrases now, even though unused this phase	The detection logic and the platform-specific wording are two different things to get right; building both together now means Phase 3 only has to point this function at a real page, not rewrite it.
One shared browser process, isolated contexts — not 3 separate browser processes	Equivalent isolation, far less memory/startup overhead. Standard Playwright pattern for multi-account automation.
<code>build_daily_schedule()</code> built but not run continuously yet	No permanent background process exists in local development. The function is real and tested for syntax/import correctness; it activates for real in Phase 10 when the agent moves to an always-on server.

7. What you should do right now

1 Try the manual trigger yourself

```
agent/venv/Scripts/python.exe -m agent.scheduler
```

Expect all 3 accounts to report `OK` — same result shown in section 3. Since it's a forced test run, it works regardless of the actual scheduled time.

2 Commit and push

```
git add agent/db/repositories.py agent/core/account_pool.py agent/core/session.py agent/core/health.py
agent/scheduler.py docs/PHASE_2_EXPLAINED.html docs/PHASE_2_EXPLAINED.pdf docs/PHASE_2_SUMMARY.html
docs/PHASE_2_SUMMARY.pdf PROGRESS.md REQUIREMENTS_COVERAGE.md
git commit -m "Phase 2 – account pool, isolated browser sessions, health checks, scheduler"
git push
```

3 Say the word for Phase 3

Phase 3 is discovery — the agent's first real interaction with LinkedIn and Instagram. This is where the health-check phrasing built in this phase gets its first real exercise.

8. What's needed before Phase 3

Nothing new required to start. Phase 3 needs the 3 accounts' actual LinkedIn/Instagram login details when it comes time to test real discovery — that conversation happens at the start of that phase, not before.

Still open for later phases: a Claude API key (Phase 4) and the WhatsApp provider decision (Phase 7).

Phase 2 of 11 complete and verified live · Generated 27 July 2026

Source: `docs/PHASE_2_EXPLAINED.html` · Rendered by `tools/make_pdf.py`